

A decorative frame composed of two thick, light-colored L-shaped lines. One L-shape is positioned in the top-left corner, and the other is in the bottom-right corner, creating an open rectangular frame around the central text.

SOAP-ИНЪЕКЦИИ

SOAP — Введение

SOAP — это сокращение от Simple Object Access Protocol. Он определен Консорциумом World Wide Web.

SOAP — это легкий протокол для обмена информацией в децентрализованной распределенной среде. Это протокол на основе XML, который состоит из трех частей: конверт, который определяет структуру для описания того, что находится в сообщении и как его обрабатывать; набор правил кодирования для выражения экземпляров определяемых приложением типов данных; и соглашение для представления удаленных вызовов процедур и ответов.

SOAP — Важные функции

Ниже приведены некоторые важные функции SOAP.

- Это коммуникационный протокол, предназначенный для общения через Интернет.
- Это может расширить HTTP для обмена сообщениями XML.
- Он обеспечивает передачу данных для веб-сервисов.
- Он может обмениваться полными документами или вызывать удаленную процедуру.
- Может использоваться для трансляции сообщения.
- Он не зависит от платформы и языка.
- Это XML-способ определения, какая информация отправляется и как.
- Это позволяет клиентским приложениям легко подключаться к удаленным службам и вызывать удаленные методы.

SOAP — Сообщения

SOAP-сообщение — это обычный XML-документ, содержащий следующие элементы:

- Конверт — определяет начало и конец сообщения. Это обязательный элемент.
- Заголовок — содержит любые необязательные атрибуты сообщения, используемые при обработке сообщения, либо в промежуточной точке, либо в конечной конечной точке. Это необязательный элемент.
- Тело — содержит данные XML, содержащие отправляемое сообщение. Это обязательный элемент.
- Неисправность — необязательный элемент неисправности, который предоставляет информацию об ошибках, возникающих при обработке сообщения.

SOAP — структура сообщения

```
<?xml version = "1.0"?>
<SOAP-ENV:Envelope
xmlns:SOAP-ENV = "http://www.w3.org/2001/12/soap-envelope"
SOAP-ENV:encodingStyle = "http://www.w3.org/2001/12/soap-encoding">
  <SOAP-ENV:Header>
    ...
  </SOAP-ENV:Header>
  <SOAP-ENV:Body>
    ...
    <SOAP-ENV:Fault>
      ...
    </SOAP-ENV:Fault>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Преимущества и недостатки

Преимущества

- Веб-службы обеспечивают взаимодействие программных систем независимо от платформы. Например, Windows-C#-клиент может обмениваться данными с Java-сервером, работающим под Linux.
- Веб-службы основаны на базе открытых стандартов и протоколов. Благодаря использованию XML достигается простота разработки и отладки веб-служб.
- Использование интернет-протокола обеспечивает HTTP-взаимодействие программных систем через межсетевой экран. Это значительное преимущество, по сравнению с такими технологиями, как CORBA, DCOM или Java RMI. С другой стороны, веб-службы не привязаны намертво к HTTP — могут использоваться и другие протоколы.

Преимущества и недостатки

Недостатки

- Меньшая производительность и больший размер сетевого трафика по сравнению с технологиями RMI, CORBA, DCOM за счёт использования текстовых XML-сообщений. Однако на некоторых веб-серверах возможна настройка сжатия сетевого трафика.
- Аспекты безопасности. Ответственные веб-службы должны использовать кодирование, возможно — требовать аутентификации пользователя. Достаточно ли здесь применения HTTPS, или предпочтительны такие решения, как XML Signature, XML Encryption или SAML — должно быть решено разработчиком.

SOAP уязвимости

SOAP, как известно, в своем фундаменте имеет XML и основные уязвимости вытекают отсюда.

SOAP injection — самая простая, но тяжело детектируемая атака, непосредственно инъекция в теле SOAP запроса.

Пример запроса

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:web="web:">
  <soapenv:Header>
  </soapenv:Header>
  <soapenv:Body>
    <web1:Login xmlns:web1="http://ws.example.com/">
      <fname>Ivan</fname>
      <lname>Ivanov</lname>
      <password>Imao1337</password>
    </web1:Login>
  </soapenv:Body>
</soapenv:Envelope>
```

Пример запроса

Это классический запрос авторизации, и если он пройдет успешно, мы получим ответ ОК. Но теперь уберем тэг `<Iname></Iname>` из запроса. В ответ от сервера получим, что-то вроде:

```
Error at line 66: Iname == null | loginid
```

```
loginid cannot be null
```

Пример запроса

Вообще данный ответ содержит в себе кусок кода, и говорит нам о том, что если `Iname` тэг отсутствует, то должен использоваться `loginid` вместо него. Получается, если мы создадим запрос, в котором укажем `loginid` без строки `<Iname>`:

```
<fname>Ivan</fname>
```

```
<password>Imao1337</password>
```

```
<loginid>1</loginid>
```

SOAP Action

Допустим, у нас есть приложение, уязвимое к SOAPAction — спуфингу по двум операциям: “createUser” и “deleteAllUsers”. При этом, перед приложением стоит шлюз, который блокирует все запросы на удаление всех пользователей, если они пришли извне (deleteAllUsers), т.е. только прямое подключение к приложению позволяет авторизованному пользователю выполнить удаление.

Пример запроса пользователя на создание нового пользователя:

Теперь, если злоумышленник добавит в запрос заголовок SOAPAction:

POST /service HTTP/1.1

Host: myHost

SOAPAction: "deleteAllUsers"

<Envelope>

<Header />

<Body>

<createUser>

<login>IVANPOPOLAM</login>

<pwd>secret</pwd>

</createUser>

</Body>

</Envelope>

Пример запроса пользователя на создание нового пользователя:

Шлюз, стоящий перед приложением пропустит такой запрос, т.к. он смотрит только в тело SOAP и в следствии приложение прочитав заголовок SOAPAction удалит всех пользователей.

Защита

Data Guard — маскирование критических данных в запросах. Например, если в API передаются серия и номер паспорта, есть возможность маскировать эту информацию.

Защита

Parameter parsing — автоматический парсинг параметров из запроса для создания отдельных логических объектов. Далее созданные объекты могут быть индивидуально настроены для применения тех или иных ограничений.

Защита

XML Content — в системе реализовать функционал защиты XML именно по сигнатурному анализу, максимально разрешенная глубина вложений, количество параметров, размер.